

Modul 2: Backpropagation dan Automatic Differentiation

Nama: Fadil Alfarizzi

NIM: 123450048

Kelas: RC

Tanggal: 2026-09-26

Simpan berkas ini sebagai `M02_NIM.ipynb` sebelum mulai mengerjakan.

Petunjuk

1. Ganti seluruh penanda `TOD0` . Jangan menghapus sel pemeriksaan.
2. Nilai Kasus 1 tidak boleh diubah; seluruh angka pada modul mengacu padanya.
3. Gunakan `float64` untuk semua perhitungan gradien.
4. Tuliskan turunan manual pada sel markdown, bukan hanya di kertas.
5. Notebook harus lolos *Restart Kernel and Run All* sebelum dikumpulkan.
6. Luaran: `M02_NIM.ipynb` , `M02_NIM.pdf` , dan `M02_NIM_metrics.csv` .

```
import platform
import random

import numpy as np
import pandas as pd
import torch
from torch import nn

NIM = '123450048' # contoh: '120450123'
SEED = int(str(NIM)[-4:]) if str(NIM).isdigit() else 42 # seed individual
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

seed_everything(SEED)
torch.set_default_dtype(torch.float64)
pd.set_option('display.precision', 8)
print({'python': platform.python_version(), 'numpy': np.__version__,
      'torch': torch.__version__, 'device': str(DEVICE), 'seed': SEED})

{'python': '3.11.15', 'numpy': '2.4.4', 'torch': '2.14.0+cu130', 'device': 'cpu', 'seed': 48}
```

A. Pre-lab - 10 poin

Jawab sebelum sesi praktikum dimulai.

1. **Gradien lokal vs gradien total pada satu simpul:** gradien lokal adalah turunan keluaran simpul terhadap masukannya sendiri. Gradien total dari loss terhadap simpul diperoleh dengan aturan rantai, yaitu gradien dari bagian sesudah simpul (upstream) dikalikan gradien lokal simpul.
2. **Mengapa `backward()` hanya dapat dipanggil pada tensor skalar:** backward membutuhkan satu nilai gradien awal, yaitu $\partial \mathcal{L} / \partial \mathcal{L} = 1$. Untuk tensor non-skalar,

turunannya berupa Jacobian sehingga harus diberikan vektor gradien eksternal (`gradient=`) yang ukurannya sama.

3. Isi `.grad` bila `backward()` dipanggil dua kali tanpa `zero_grad()` : PyTorch menjumlahkan gradien baru ke `.grad` yang lama, sehingga untuk graf dan input yang sama nilainya menjadi dua kali lipat.
4. **Mengapa turunan BCE-with-logits terhadap logit berbentuk $p - y$, bukan $-y/p$** : BCE-with-logits menggabungkan sigmoid dan BCE. Dengan $\mathcal{L} = -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z))$ dan $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, faktor $p(1 - p)$ saling menghapus sehingga $\partial \mathcal{L} / \partial z = p - y$. Bentuk $-y/p$ adalah turunan terhadap probabilitas p , bukan terhadap logit, dan tidak stabil saat $p \rightarrow 0$.

Graf komputasi Kasus 1. Tuliskan urutan simpul dari $\mathbf{x}$ sampai $\mathcal{L}$, lalu tandai gradien lokal di setiap simpul:

$$\mathbf{x} \rightarrow \mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \rightarrow \mathbf{h} = \text{ReLU}(\mathbf{z}^{(1)}) \rightarrow \mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h} + b^{(2)} \rightarrow p = \sigma(\mathbf{z}^{(2)}) \rightarrow \mathcal{L}$$

Gradien lokal: $\partial \mathbf{z}^{(1)} / \partial \mathbf{W}^{(1)} = \mathbf{x}^T$, $\partial h_i / \partial z_i^{(1)} = 1[z_i^{(1)} > 0]$, $\partial z^{(2)} / \partial \mathbf{h} = \mathbf{W}^{(2)}$, $\partial z^{(2)} / \partial \mathbf{W}^{(2)} = \mathbf{h}$, dan gabungan sigmoid-BCE $\partial \mathcal{L} / \partial z^{(2)} = p - y$.

B. Turunan manual - 20 poin

Kasus 1 memakai nilai tetap berikut:

$$\mathbf{x} = [2 \quad -1], \quad \mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = [0 \quad 0], \quad \mathbf{W}^{(2)} = [2 \quad -1], \quad b^{(2)} = 0.5,$$

Tuliskan penurunan Anda **berurutan** di sini, satu baris satu langkah:

1. $\partial \mathcal{L} / \partial z^{(2)} = p - y = \sigma(2,5) - 1 = 0,92414182 - 1 = -0,07585818$
2. $\partial \mathcal{L} / \partial \mathbf{W}^{(2)} = (p - y) \mathbf{h} = -0,07585818 \times [1,5 \quad 1,0] = [-0,11378727 \quad -0,07585818]$
3. $\partial \mathcal{L} / \partial b^{(2)} = p - y = -0,07585818$
4. $\partial \mathcal{L} / \partial \mathbf{h} = (p - y) \mathbf{W}^{(2)} = -0,07585818 \times [2 \quad -1] = [-0,15171636 \quad 0,07585818]$
5. $\partial \mathcal{L} / \partial \mathbf{z}^{(1)} = \partial \mathcal{L} / \partial \mathbf{h} \odot 1[\mathbf{z}^{(1)} > 0]$; karena $\mathbf{z}^{(1)} = [1,5 \quad 1,0]$ keduanya positif, hasilnya tetap $[-0,15171636 \quad 0,07585818]$
6. $\partial \mathcal{L} / \partial \mathbf{W}^{(1)} = (\partial \mathcal{L} / \partial \mathbf{z}^{(1)})^T \mathbf{x}$ (perkalian luar) = $\begin{bmatrix} -0,30343272 & 0,15171636 \\ 0,15171636 & -0,07585818 \end{bmatrix}$
7. $\partial \mathcal{L} / \partial \mathbf{b}^{(1)} = \partial \mathcal{L} / \partial \mathbf{z}^{(1)} = [-0,15171636 \quad 0,07585818]$

Cantumkan pula shape setiap gradien: $\partial \mathcal{L} / \partial \mathbf{W}^{(1)}$ berukuran (2, 2), $\partial \mathcal{L} / \partial \mathbf{b}^{(1)}$ berukuran (2,), $\partial \mathcal{L} / \partial \mathbf{W}^{(2)}$ berukuran (2,), dan $\partial \mathcal{L} / \partial b^{(2)}$ skalar. Setiap gradien selalu berukuran sama dengan parameternya.

```
x = np.array([2.0, -1.0])
W1 = np.array([[0.5, -0.5], [1.0, 1.0]])
b1 = np.array([0.0, 0.0])
W2 = np.array([2.0, -1.0])
b2 = 0.5
y = 1.0
```

```
def forward(x, W1, b1, W2, b2, y):
    """Kembalikan dict berisi z1, h, z2, p, dan loss."""
    z1 = W1 @ x + b1
    h = np.maximum(z1, 0.0)
    z2 = W2 @ h + b2
    p = 1.0 / (1.0 + np.exp(-z2))
    loss = np.logaddexp(0.0, z2) - y * z2 # BCE-with-logits yang stabil
    return {'z1': z1, 'h': h, 'z2': z2, 'p': p, 'loss': loss}
```

```
nilai = forward(x, W1, b1, W2, b2, y)
print({k: np.round(v, 6) for k, v in nilai.items()})
```

```
# Pemeriksaan wajib: jangan diubah.
assert np.allclose(nilai['z1'], [1.5, 1.0]), 'z1 belum benar'
assert np.isclose(nilai['z2'], 2.5), 'logit belum benar'
assert np.isclose(nilai['loss'], 0.0788897, atol=1e-6), 'loss belum benar'
print('forward pass sesuai Kasus 1')
```

```
{'z1': array([1.5, 1. ]), 'h': array([1.5, 1. ]), 'z2': np.float64(2.5), 'p': np.float64(0.924142), 'loss': np.float64(0.07889)}
forward pass sesuai Kasus 1
```

```
def backward(x, W1, W2, y, nilai):
    """Kembalikan dict gradien untuk 'W1', 'b1', 'W2', 'b2'.


Urutan pengerjaan: dz2 -> (dW2, db2, dh) -> dz1 -> (dW1, db1).



Ingat gradien lokal ReLU dan bentuk perkalian luar untuk dW1.



"""


    dz2 = nilai['p'] - y
    dW2 = dz2 * nilai['h']
    db2 = dz2
    dh = dz2 * W2
    dz1 = dh * (nilai['z1'] > 0.0)
    dW1 = np.outer(dz1, x)
    db1 = dz1
    return {'W1': dW1, 'b1': db1, 'W2': dW2, 'b2': db2}
```

```
grad_manual = backward(x, W1, W2, y, nilai)
for nama, v in grad_manual.items():
    print(f'{nama:>3}: {np.round(v, 7)}')
```

```
# Pemeriksaan wajib: dua angka kunci dari modul.
assert np.allclose(grad_manual['W2'], [-0.1137873, -0.0758582], atol=1e-6)
assert grad_manual['W1'].shape == W1.shape, 'shape dW1 harus sama dengan W1'
print('gradien manual sesuai angka acuan')
```

```
W1: [[-0.3034327  0.1517164]
      [ 0.1517164 -0.0758582]]
b1: [-0.1517164  0.0758582]
W2: [-0.1137873 -0.0758582]
b2: -0.0758582
gradien manual sesuai angka acuan
```

C. Autograd - 20 poin

Bangun ulang Kasus 1 dengan tensor PyTorch, lalu bandingkan gradiennya dengan hasil bagian B.

```
tw1 = torch.tensor(W1, requires_grad=True)
tb1 = torch.tensor(b1, requires_grad=True)
tw2 = torch.tensor(W2, requires_grad=True)
tb2 = torch.tensor(b2, requires_grad=True)
tx, ty = torch.tensor(x), torch.tensor(y)
kriteria = nn.BCEWithLogitsLoss()

def forward_torch():
    """Hitung loss dengan BCEWithLogitsLoss pada LOGIT."""
    z1 = tw1 @ tx + tb1
    h = torch.relu(z1)
    z2 = tw2 @ h + tb2
    return kriteria(z2, ty)

loss = forward_torch()
loss.backward()

for nama, t in [('W1', tw1), ('b1', tb1), ('W2', tw2), ('b2', tb2)]:
    selisih = np.max(np.abs(t.grad.numpy() - grad_manual[nama]))
    print(f'{nama}: autograd = {np.round(t.grad.numpy(), 7)} selisih maks = {selisih:.2e}')
    assert selisih < 1e-10, f'gradien {nama} belum cocok dengan hasil manual'
print('autograd cocok dengan backward manual')
```

```

W1: autograd = [[-0.3034327  0.1517164]
 [ 0.1517164 -0.0758582]] selisih maks = 0.00e+00
b1: autograd = [-0.1517164  0.0758582] selisih maks = 0.00e+00
W2: autograd = [-0.1137873 -0.0758582] selisih maks = 0.00e+00
b2: autograd = -0.0758582 selisih maks = 0.00e+00
autograd cocok dengan backward manual

```

```

# Panggil backward() sekali lagi TANPA menghapus gradien,
# cetak tw2.grad, lalu hapus gradien dan hitung ulang.
# Penjelasan hasilnya ada pada sel markdown di bawah.
grad_pertama = tw2.grad.detach().clone()
loss = forward_torch()
loss.backward()
print('tw2.grad setelah backward pertama:', grad_pertama.numpy())
print('tw2.grad setelah backward kedua  :', tw2.grad.numpy())
print('rasio                               :', (tw2.grad / grad_pertama).numpy())

for t in (tw1, tb1, tw2, tb2):
    t.grad = None
loss = forward_torch()
loss.backward()
print('tw2.grad setelah dihapus lalu dihitung ulang:', tw2.grad.numpy())
assert torch.allclose(tw2.grad, grad_pertama)

```

```

tw2.grad setelah backward pertama: [-0.11378727 -0.07585818]
tw2.grad setelah backward kedua  : [-0.22757454 -0.15171636]
rasio                               : [2.  2.]
tw2.grad setelah dihapus lalu dihitung ulang: [-0.11378727 -0.07585818]

```

Penjelasan akumulasi gradien: `backward()` kedua **menambahkan** gradien baru ke `.grad` yang sudah ada, sehingga `tw2.grad` menjadi tepat **dua kali** gradien satu lintasan ($[-0,22757454 \quad -0,15171636]$). Setelah gradien dihapus (`grad = None`) lalu dihitung ulang, nilainya kembali ke $[-0,11378727 \quad -0,07585818]$. Itulah peran `optimizer.zero_grad()` pada training loop: tanpa pemanggilan ini, gradien setiap batch menumpuk dari langkah-langkah sebelumnya sehingga update parameter menjadi salah.

(Sebutkan berapa kali lipat nilainya dan hubungkan dengan peran `optimizer.zero_grad()` pada training loop.)

D. Gradient checking - 20 poin

Bandingkan gradien analitik dengan selisih terpusat:

$$g_{\text{num}} = \frac{\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)}{2\epsilon}, \quad \text{rel err} = \frac{|g_{\text{analitik}} - g_{\text{num}}|}{|g_{\text{analitik}}| + |g_{\text{num}}| + 10^{-12}}$$

Ambang lulus: seluruh baris di bawah 10^{-5} .

```

EPS = 1e-5

def loss_dengan(param, i, delta):
    """Salin parameter, geser satu komponen sebesar delta, kembalikan loss."""
    params = {'W1': W1.copy(), 'b1': b1.copy(), 'W2': W2.copy(), 'b2': float(b2)}
    if i == ():
        params[param] += delta
    else:
        params[param][i] += delta
    return forward(x, params['W1'], params['b1'], params['W2'], params['b2'], y)['loss']

def finite_difference(param, i):
    """Kembalikan gradien numerik dengan selisih terpusat."""
    return (loss_dengan(param, i, EPS) - loss_dengan(param, i, -EPS)) / (2.0 * EPS)

indeks = (('W1', (0, 0)), ('W1', (0, 1)), ('W1', (1, 0)), ('W1', (1, 1)))
        + (('b1', (0,)), ('b1', (1,)))
        + (('W2', (0,)), ('W2', (1,)))
        + (('b2', ()),)

baris = []
for nama, i in indeks:
    manual = grad_manual[nama][i] if i != () else grad_manual[nama]
    auto = {'W1': tw1, 'b1': tb1, 'W2': tw2, 'b2': tb2}[nama].grad.numpy()
    auto = auto[i] if i != () else auto

```

```

numerik = finite_difference(nama, i)
rel = abs(manual - numerik) / (abs(manual) + abs(numerik) + 1e-12)
baris.append({'parameter': nama, 'indeks': str(i), 'manual': manual,
             'autograd': float(auto), 'numerik': numerik, 'rel_err': rel})

tabel = pd.DataFrame(baris)
print(tabel.to_string(index=False))
print('\nrelative error maksimum:', tabel['rel_err'].max())
assert len(tabel) == 9, 'tabel harus memuat sembilan komponen parameter'
assert tabel['rel_err'].max() < 1e-5, 'masih ada baris yang melampaui ambang'

```

parameter	indeks	manual	autograd	numerik	rel_err
W1	(0, 0)	-0.30343272	-0.30343272	-0.30343272	1.07313401e-10
W1	(0, 1)	0.15171636	0.15171636	0.15171636	3.41358596e-11
W1	(1, 0)	0.15171636	0.15171636	0.15171636	3.41358596e-11
W1	(1, 1)	-0.07585818	-0.07585818	-0.07585818	1.12219224e-10
b1	(0,)	-0.15171636	-0.15171636	-0.15171636	3.41358596e-11
b1	(1,)	0.07585818	0.07585818	0.07585818	1.12219224e-10
W2	(0,)	-0.11378727	-0.11378727	-0.11378727	8.29208263e-11
W2	(1,)	-0.07585818	-0.07585818	-0.07585818	1.12219224e-10
b2	(,)	-0.07585818	-0.07585818	-0.07585818	1.12219224e-10

relative error maksimum: 1.1221922358447185e-10

```

# Simpan tabel ke M02_NIM_metrics.csv.
tabel.insert(0, 'run_id', 'gradcheck')
tabel.insert(1, 'seed', SEED)
tabel.to_csv(f'M02_{NIM}_metrics.csv', index=False)
print('tersimpan')

```

tersimpan

Checkpoint menit ke-95. Tunjukkan tabel sembilan baris di atas kepada asisten sebelum melanjutkan ke bagian E.

E. Diagnosis training loop - 20 poin

Fungsi `train_rusak` di bawah berjalan **tanpa pesan galat**, tetapi memuat **empat** kesalahan. Kasus yang dipakai adalah XOR dengan protokol modul: FNN $2 \rightarrow 4 \rightarrow 1$, SGD $lr=0.1$, 400 epoch, satu batch penuh.

```

X_xor = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_xor = torch.tensor([[0.0], [1.0], [1.0], [0.0]])

def train_rusak(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)
    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.Linear(4, 1),
    )
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        logits = model(X_xor)
        loss = kriteria(torch.sigmoid(logits), y_xor)
        opt.step()
        loss.backward()
        riwayat.append(loss.item())
    return model, riwayat

# Catatan: pada PyTorch versi yang dipakai, urutan opt.step() -> loss.backward()
# memicu RuntimeError pada epoch kedua, sehingga pemanggilan dibungkus try/except.
try:
    model_rusak, riwayat_rusak = train_rusak()
    print(f'loss awal : {riwayat_rusak[0]:.4f}')
    print(f'loss akhir : {riwayat_rusak[-1]:.4f}')
    with torch.no_grad():
        print('prediksi :', (torch.sigmoid(model_rusak(X_xor)) > 0.5).int().flatten().tolist())
        print('target :', y_xor.int().flatten().tolist())
except RuntimeError as galat:
    print('train_rusak berhenti dengan RuntimeError:')
    print(' ', str(galat).split('. Hint')[0])

```

train_rusak berhenti dengan RuntimeError:
 one of the variables needed for gradient computation has been modified by an inplace operation: [torch.DoubleTensor [4, 1]], which is output 0 of AsStrided, is at version 2; expected version 1 instead

Temuan kesalahan

Isi tabel berikut. Setiap baris harus menyebut **baris kode**, **alasan**, dan **gejala** yang teramati.

No	Baris kode bermasalah	Mengapa keliru	Gejala yang terlihat
1	<code>nn.Linear(2, 4)</code> langsung diikuti <code>nn.Linear(4, 1)</code> tanpa aktivasi	Komposisi dua layer affine tetap fungsi linear, sehingga batas keputusannya satu garis lurus yang tidak dapat memisahkan XOR.	Selama ReLU belum ditambahkan (perbaikan-1 dan 2), prediksi tetap <code>[0, 0, 0, 0]</code> dan loss tertahan di sekitar 0,69 (setara tebakan acak $\ln 2$).
2	<code>loss = kriteria(torch.sigmoid(logits), y_xor)</code>	<code>BCEWithLogitsLoss</code> sudah menerapkan sigmoid di dalamnya. Sigmoid ganda membuat model menghitung $\sigma(\sigma(z)) \in (0,5; 0,73)$, sehingga probabilitas tidak pernah bisa di bawah 0,5 dan gradien terhadap logit mengecil.	Setelah tiga perbaikan lain (perbaikan-3), loss hanya turun dari 0,7268 ke 0,6975 dan prediksi masih <code>[0, 0, 0, 0]</code> . Loss awal baru menjadi 0,6954 ($\approx \ln 2$) setelah sigmoid ganda dibuang.
3	<code>opt.step()</code> dipanggil sebelum <code>loss.backward()</code>	Parameter diperbarui memakai gradien lama (atau kosong pada epoch pertama) sebelum gradien loss saat ini dihitung. Update in-place itu juga mengubah tensor yang masih dibutuhkan graf komputasi.	Pada PyTorch 2.14, <code>train_rusak</code> berhenti di epoch kedua dengan <code>RuntimeError: one of the variables needed for gradient computation has been modified by an inplace operation</code> . Pada versi lama yang tidak memeriksa ini, setiap update tertinggal satu langkah.
4	Tidak ada <code>opt.zero_grad()</code> di dalam loop	Gradien PyTorch terakumulasi di <code>.grad</code> (lihat bagian C), sehingga setiap step memakai jumlah gradien semua epoch sebelumnya, bukan gradien epoch saat ini.	Pada perbaikan-1 (urutan benar, tanpa <code>zero_grad</code>), norma <code>.grad</code> membesar dari 0,067 ke 0,935 dan norma bobot mencapai 36,4 (dibanding 1,9 dengan <code>zero_grad</code>). Keluaran menjadi jenuh, loss terkunci tepat di 0,6931 dan prediksi <code>[0, 0, 0, 0]</code> .

```
# Perbaiki SATU PER SATU. Setiap tahap menyalin train_rusak dan memperbaiki
# satu kesalahan tambahan, lalu mencatat loss akhirnya.
PERBAIKAN = [
    'urutan: loss.backward() sebelum opt.step()',
    'menambahkan opt.zero_grad() tiap epoch',
    'menambahkan ReLU di antara dua Linear',
    'BCEWithLogitsLoss langsung pada logit (tanpa sigmoid ganda)',
]
```

```
def train_tahap(k: int, epoch: int = 400, lr: float = 0.1):
    """train_rusak dengan k perbaikan pertama dari daftar PERBAIKAN."""
    seed_everything(SEED)
    lapis = [nn.Linear(2, 4)]
    if k >= 3:
        lapis.append(nn.ReLU())
    lapis.append(nn.Linear(4, 1))
    model = nn.Sequential(*lapis)
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []
    for _ in range(epoch):
        if k >= 2:
            opt.zero_grad()
            logits = model(X_xor)
            loss = kriteria(logits) if k >= 4 else torch.sigmoid(logits), y_xor
            loss.backward() # k >= 1: backward sebelum step
            opt.step()
            riwayat.append(loss.item())
        with torch.no_grad():
            prediksi = (torch.sigmoid(model(X_xor)) > 0.5).int().flatten()
            benar = int((prediksi == y_xor.int().flatten()).sum())
    return model, riwayat, benar, prediksi.tolist()

tahap = [{'tahap': 'rusak', 'yang_diperbaiki': '-', 'loss_awal': np.nan,
            'loss_akhir': np.nan, 'benar': np.nan, 'prediksi': 'RuntimeError'}]
for k in range(1, 5):
    _, riw, benar, pred = train_tahap(k)
    tahap.append({'tahap': f'perbaikan-{k}', 'yang_diperbaiki': PERBAIKAN[k - 1],
                  'loss_awal': riw[0], 'loss_akhir': riw[-1], 'benar': benar,
                  'prediksi': str(pred)})
    print(f'perbaikan-{k}: loss {riw[0]:.4f} -> {riw[-1]:.4g}, benar {benar}/4, prediksi {pred}')
```

```
perbaikan-1: loss 0.7368 -> 0.6931, benar 2/4, prediksi [0, 0, 0, 0]
perbaikan-2: loss 0.7368 -> 0.6982, benar 2/4, prediksi [0, 0, 0, 0]
perbaikan-3: loss 0.7268 -> 0.6975, benar 2/4, prediksi [0, 0, 0, 0]
perbaikan-4: loss 0.6954 -> 0.6935, benar 2/4, prediksi [1, 0, 1, 0]
```

```
def train_benar(epoch: int = 400, lr: float = 0.1):
    """Versi yang sudah bebas dari keempat kesalahan.

    Selain keempat perbaikan, bobot layer tersembunyi diinisialisasi dengan
    He/Kaiming uniform (bias nol). Dengan inisialisasi bawaan PyTorch, seed 48
    tersangkut di loss sekitar 0,69 (lihat perbaikan-4 di atas).
    """
    seed_everything(SEED)
    model = nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1))
    nn.init.kaiming_uniform_(model[0].weight, nonlinearity='relu')
    nn.init.zeros_(model[0].bias)
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []
    for _ in range(epoch):
        opt.zero_grad()
        loss = kriteria(model(X_xor), y_xor)
        loss.backward()
        opt.step()
        riwayat.append(loss.item())
    return model, riwayat

model_benar, riwayat_benar = train_benar()
print(f'loss akhir: {riwayat_benar[-1]:.4f}')
with torch.no_grad():
    prediksi = (torch.sigmoid(model_benar(X_xor)) > 0.5).int().flatten()
print('prediksi :', prediksi.tolist())

assert riwayat_benar[-1] < 0.1, 'loss akhir harus di bawah 0,1'
assert torch.equal(prediksi, y_xor.int().flatten()), 'keempat titik XOR harus benar'
print('training loop sudah benar')
```

```
loss akhir: 0.0916
prediksi : [0, 1, 1, 0]
training loop sudah benar
```

```
# Gabungkan catatan tahap perbaikan ke metrics.csv.
with torch.no_grad():
    benar_final = int((prediksi == y_xor.int().flatten()).sum())
    tahap.append({'tahap': 'final', 'yang_diperbaiki': 'empat perbaikan + inisialisasi Kaiming',
                  'loss_awal': riwayat_benar[0], 'loss_akhir': riwayat_benar[-1],
                  'benar': benar_final, 'prediksi': str(prediksi.tolist())})
df_tahap = pd.DataFrame(tahap)
df_tahap.insert(0, 'seed', SEED)
df_tahap.to_csv(f'M02_{NIM}_metrics_loop.csv', index=False)
print(df_tahap.to_string(index=False))
```

seed	tahap	yang_diperbaiki	loss_awal	loss_akhir	benar	prediksi
48	rusak	-	NaN	NaN	NaN	RuntimeError
48	perbaikan-1	urutan: loss.backward() sebelum opt.step()	0.73680427	0.69314718	2.0	[0, 0, 0, 0]
48	perbaikan-2	menambahkan opt.zero_grad() tiap epoch	0.73680427	0.69820487	2.0	[0, 0, 0, 0]
48	perbaikan-3	menambahkan ReLU di antara dua Linear	0.72676558	0.69747868	2.0	[0, 0, 0, 0]
48	perbaikan-4	BCEWithLogitsLoss langsung pada logit (tanpa sigmoid ganda)	0.69542796	0.69346850	2.0	[1, 0, 1, 0]
48	final	empat perbaikan + inisialisasi Kaiming	0.73122009	0.09162081	4.0	[0, 1, 1, 0]

F. Tugas individu

Kerjakan ketiganya di sel-sel baru di bawah bagian ini.

1. **Perluasan jaringan.** Tambahkan neuron ketiga pada hidden layer: baris $[-1 \ 0.5]$ pada $\mathbf{W}^{(1)}$, bias 0,25, dan komponen $-0,5$ pada $\mathbf{W}^{(2)}$. Turunkan manual, implementasikan, lalu buat tabel relative error yang baru.
2. **Batch dua contoh.** Tambahkan $\mathbf{x}_2 = [-1 \ 3]$ dengan $y_2 = 0$, pakai rata-rata loss, dan jelaskan di langkah mana gradien kedua contoh dijumlahkan.
3. **Laporan diagnosis.** Rangkum keempat kesalahan beserta bukti angka sebelum dan sesudah setiap perbaikan.

F1. Perluasan jaringan

Neuron ketiga: baris $[-1 \ 0,5]$ pada $\mathbf{W}^{(1)}$, bias 0,25, dan komponen $-0,5$ pada $\mathbf{W}^{(2)}$.

Turunan manual: $z_3^{(1)} = -1 \cdot 2 + 0,5 \cdot (-1) + 0,25 = -2,25 < 0$, sehingga $h_3 = 0$. Akibatnya $z^{(2)}$, p , dan loss **tidak berubah** ($z^{(2)} = 2,5$, $\mathcal{L} = 0,0788897$). Rumus backprop tetap sama: $\partial \mathcal{L} / \partial h_3 = (p - y)(-0,5) = 0,03792909$, tetapi gradien lokal ReLU pada neuron ketiga bernilai 0, sehingga $\partial \mathcal{L} / \partial z_3^{(1)} = 0$, baris ketiga $\partial \mathcal{L} / \partial \mathbf{W}^{(1)}$ bernilai $[0 \ 0]$, $\partial \mathcal{L} / \partial b_3^{(1)} = 0$, dan $\partial \mathcal{L} / \partial W_3^{(2)} = (p - y)h_3 = 0$. Gradien parameter lain sama dengan Kasus 1.

```
W1_ext = np.vstack([W1, [-1.0, 0.5]])
b1_ext = np.array([0.0, 0.0, 0.25])
W2_ext = np.array([2.0, -1.0, -0.5])
nilai_ext = forward(x, W1_ext, b1_ext, W2_ext, b2, y)
grad_ext = backward(x, W1_ext, W2_ext, y, nilai_ext)
print({k: np.round(v, 6) for k, v in nilai_ext.items()})

def fd_ext(param, i, eps=EPS):
    def loss_geser(delta):
        p = {'W1': W1_ext.copy(), 'b1': b1_ext.copy(), 'W2': W2_ext.copy(), 'b2': float(b2)}
        if i == ():
            p[param] += delta
        else:
            p[param][i] += delta
        return forward(x, p['W1'], p['b1'], p['W2'], p['b2'], y)['loss']
    return (loss_geser(eps) - loss_geser(-eps)) / (2 * eps)

baris_ext = []
for nama, g in grad_ext.items():
    for i in (np.ndindex(np.shape(g)) if np.ndim(g) else [()]):
        manual = g[i] if i != () else g
        numerik = fd_ext(nama, i)
        rel = abs(manual - numerik) / (abs(manual) + abs(numerik) + 1e-12)
        baris_ext.append({'parameter': nama, 'indeks': str(i), 'manual': manual,
                        'numerik': numerik, 'rel_err': rel})

tabel_ext = pd.DataFrame(baris_ext)
print(tabel_ext.to_string(index=False))
print('\nrelative error maksimum:', tabel_ext['rel_err'].max())
assert tabel_ext['rel_err'].max() < 1e-5
```



```
{'z1': array([ 1.5 ,  1. , -2.25]), 'h': array([1.5, 1. , 0. ]), 'z2': np.float64(2.5), 'p': np.float64(0.924142), 'loss': np.float64(0.07889)}
```

parameter	indeks	manual	numerik	rel_err
W1	(0, 0)	-0.30343272	-0.30343272	1.07313401e-10
W1	(0, 1)	0.15171636	0.15171636	3.41358596e-11
W1	(1, 0)	0.15171636	0.15171636	3.41358596e-11
W1	(1, 1)	-0.07585818	-0.07585818	1.12219224e-10
W1	(2, 0)	0.00000000	0.00000000	0.00000000e+00
W1	(2, 1)	-0.00000000	0.00000000	0.00000000e+00
b1	(0,)	-0.15171636	-0.15171636	3.41358596e-11
b1	(1,)	0.07585818	0.07585818	1.12219224e-10
b1	(2,)	0.00000000	0.00000000	0.00000000e+00
W2	(0,)	-0.11378727	-0.11378727	8.29208263e-11
W2	(1,)	-0.07585818	-0.07585818	1.12219224e-10
W2	(2,)	-0.00000000	0.00000000	0.00000000e+00
b2	()	-0.07585818	-0.07585818	1.12219224e-10

relative error maksimum: 1.1221922358447185e-10

F2. Batch dua contoh

Contoh kedua $\mathbf{x}_2 = [-1 \ 3]$, $y_2 = 0$, dan loss batch adalah rata-rata $\mathcal{L} = \frac{1}{2}(\mathcal{L}_1 + \mathcal{L}_2)$.

Forward dan backward per contoh berjalan terpisah sampai $\partial \mathcal{L} / \partial z_n^{(2)} = (p_n - y_n) / 2$.

Gradien kedua contoh **dijumlahkan pada langkah gradien parameter**, yaitu saat

menghitung $\partial \mathcal{L} / \partial \mathbf{W}^{(2)} = \sum_n \delta_n^{(2)} \mathbf{h}_n$, $\partial \mathcal{L} / \partial b^{(2)} = \sum_n \delta_n^{(2)}$, $\partial \mathcal{L} / \partial \mathbf{W}^{(1)} = \sum_n \delta_n^{(1)} \mathbf{x}_n^T$,

dan $\partial \mathcal{L} / \partial \mathbf{b}^{(1)} = \sum_n \delta_n^{(1)}$. Dalam bentuk matriks, penjumlahan ini adalah perkalian `dZ.T`

@ X dan `sum(axis=0)`.

```
X_batch = np.array([[2.0, -1.0], [-1.0, 3.0]])
y_batch = np.array([1.0, 0.0])

def loss_batch(W1_, b1_, W2_, b2_):
    Z1 = X_batch @ W1_.T + b1_
    Z2 = np.maximum(Z1, 0.0) @ W2_ + b2_
    return np.mean(np.logaddexp(0.0, Z2) - y_batch * Z2)

Z1 = X_batch @ W1.T + b1
H = np.maximum(Z1, 0.0)
Z2 = H @ W2 + b2
P = 1.0 / (1.0 + np.exp(-Z2))
dZ2 = (P - y_batch) / len(X_batch) # gradien per contoh, sudah dibagi B
grad_batch = {
    'W2': dZ2 @ H, # penjumlahan kedua contoh
    'b2': dZ2.sum(),
}
dZ1 = (dZ2[:, None] * W2) * (Z1 > 0.0)
grad_batch['W1'] = dZ1.T @ X_batch # penjumlahan kedua contoh
grad_batch['b1'] = dZ1.sum(axis=0)

print('loss per contoh:', np.logaddexp(0.0, Z2) - y_batch * Z2)
print('loss batch (rata-rata):', loss_batch(W1, b1, W2, b2))

# Cek terhadap autograd.
bW1 = torch.tensor(W1, requires_grad=True); b1 = torch.tensor(b1, requires_grad=True)
bW2 = torch.tensor(W2, requires_grad=True); b2 = torch.tensor(b2, requires_grad=True)
logit_b = torch.relu(torch.tensor(X_batch) @ bW1.T + b1) @ bW2 + b2
nn.BCEWithLogitsLoss()(logit_b, torch.tensor(y_batch)).backward()
for nama, t in [('W1', bW1), ('b1', b1), ('W2', bW2), ('b2', b2)]:
    selisih = np.max(np.abs(t.grad.numpy() - grad_batch[nama]))
    print(f'{nama}: manual = {np.round(grad_batch[nama], 8)} selisih autograd = {selisih:.1e}')
    assert selisih < 1e-10

loss per contoh: [0.07888973 0.20141328]
loss batch (rata-rata): 0.14015150613765104
W1: manual = [[-0.15171636  0.07585818]
 [ 0.16707094 -0.31156738]] selisih autograd = 0.0e+00
b1: manual = [-0.07585818 -0.05328367] selisih autograd = 0.0e+00
W2: manual = [-0.05689364  0.14449643] selisih autograd = 0.0e+00
b2: manual = 0.05328367 selisih autograd = 0.0e+00
```

F3. Laporan diagnosis

Tahap	Perbaikan	Loss awal → akhir	Prediksi (target [0, 1, 1, 0])
rusak	-	RuntimeError di epoch 2	-
perbaikan-1	backward() sebelum step()	0,7368 → 0,6931	[0, 0, 0, 0]
perbaikan-2	tambah zero_grad()	0,7368 → 0,6982	[0, 0, 0, 0]
perbaikan-3	tambah ReLU	0,7268 → 0,6975	[0, 0, 0, 0]
perbaikan-4	loss langsung pada logit	0,6954 → 0,6935	[1, 0, 1, 0]
final	+ inisialisasi Kaiming	0,7312 → 0,0916	[0, 1, 1, 0]

Keempat kesalahan: (1) urutan `step / backward` terbalik membuat loop berhenti dengan `RuntimeError`; (2) tanpa `zero_grad` gradien menumpuk sehingga bobot membesar dan keluaran jenuh di loss 0,6931; (3) tanpa ReLU model hanya linear sehingga tidak mampu memisahkan XOR; (4) sigmoid ganda membatasi probabilitas di atas 0,5 dan melemahkan gradien.

Setelah keempatnya diperbaiki, loop sudah benar secara kode, tetapi dengan inisialisasi bawaan PyTorch dan seed 48 model masih tersangkut di loss 0,6935. Dengan hanya 4 neuron ReLU, 400 epoch, dan lr 0,1, sebagian besar inisialisasi berakhir di titik ini karena beberapa neuron mati atau simetris. Karena itu `train_benar` memakai inisialisasi He/Kaiming uniform untuk layer tersembunyi. Protokol lain tetap sama (arsitektur 2-4-1, SGD lr 0,1, 400 epoch, satu batch penuh), dan hasilnya loss akhir 0,0916 dengan keempat titik XOR benar.

```
# Percobaan untuk pertanyaan G2: gradient checking dengan epsilon = 1e-9.
EPS_ASLI = EPS
hasil_eps = {}
for eps in [1e-5, 1e-9]:
    EPS = eps
    rel = []
    for nama, i in indeks:
        manual = grad_manual[nama][i] if i != () else grad_manual[nama]
        numerik = finite_difference(nama, i)
        rel.append(abs(manual - numerik) / (abs(manual) + abs(numerik) + 1e-12))
    hasil_eps[f'eps={eps:g}'] = rel
EPS = EPS_ASLI
tabel_eps = pd.DataFrame(hasil_eps, index=[f'{n}{i}' for n, i in indeks])
print(tabel_eps.to_string())
print('\nmaksimum:', tabel_eps.max().to_dict())
```

```

           eps=1e-05  eps=1e-09
W1(0, 0)  1.07313401e-10  0.00000019
W1(0, 1)  3.41358596e-11  0.00000019
W1(1, 0)  3.41358596e-11  0.00000019
W1(1, 1)  1.12219224e-10  0.00000128
b1(0,)    3.41358596e-11  0.00000019
b1(1,)    1.12219224e-10  0.00000128
W2(0,)    8.29208263e-11  0.00000030
W2(1,)    1.12219224e-10  0.00000128
b2()      1.12219224e-10  0.00000128

maksimum: {'eps=1e-05': 1.1221922358447185e-10, 'eps=1e-09': 1.2751593298349621e-06}
```

G. Pertanyaan analisis

1. Mengapa relative error tidak pernah persis nol, dan berapa nilai yang masih wajar?
Gradien numerik memakai selisih hingga, sehingga selalu ada galat pemotongan orde ϵ^2 dan galat pembulatan floating-point orde $10^{-16}/\epsilon$. Dengan `float64` dan $\epsilon = 10^{-5}$, relative error sekitar 10^{-7} atau lebih kecil sudah sangat baik, dan nilai hingga 10^{-5} masih wajar (ambang modul). Hasil di sini maksimum $1,12 \times 10^{-10}$.
2. Apa yang terjadi pada tabel bila $\epsilon = 10^{-9}$? Jalankan dan jelaskan. Dengan $\epsilon = 10^{-9}$, relative error maksimum naik dari $1,12 \times 10^{-10}$ menjadi $1,28 \times 10^{-6}$ (sekitar 10.000 kali lebih buruk). Selisih dua loss menjadi sangat kecil sehingga galat pembulatan (sekitar $10^{-16}/10^{-9} \approx 10^{-7}$) mendominasi. Tabel masih lulus ambang 10^{-5} karena memakai `float64`, tetapi dengan `float32` hasilnya akan jauh lebih buruk. Epsilon yang terlalu kecil tidak membuat pengecekan lebih akurat.
3. Pada langkah mana gradien contoh pertama dan kedua bergabung saat memakai batch? Pada langkah gradien parameter. Sampai $\partial\mathcal{L}/\partial z^{(2)}$, $\partial\mathcal{L}/\partial \mathbf{h}$, dan $\partial\mathcal{L}/\partial \mathbf{z}^{(1)}$, setiap contoh punya barisnya sendiri (masing-masing sudah dibagi $B = 2$ karena rata-rata). Keduanya baru dijumlahkan saat menghitung $\partial\mathcal{L}/\partial \mathbf{W}$ dan $\partial\mathcal{L}/\partial \mathbf{b}$, yaitu pada `dZ.T @ X` dan `sum(axis=0)` di bagian F2, karena kedua contoh memakai parameter yang sama.
4. Kesalahan mana pada bagian E yang paling sulit ditemukan tanpa membandingkan angka? Sigmoid ganda. Kode tetap berjalan tanpa galat, loss tetap terlihat turun sedikit, dan nilainya masih di kisaran wajar (sekitar 0,69-0,73), sehingga tidak ada yang tampak aneh. Kesalahan ini baru terlihat saat membandingkan angka: probabilitas tidak pernah di bawah 0,5 dan loss awal berubah dari 0,7268 ke 0,6954 setelah diperbaiki. Sebaliknya, urutan `step / backward` langsung memicu `RuntimeError`, dan tanpa ReLU atau `zero_grad` gejalanya lebih mencolok.
5. Apa beda peran backpropagation dan optimizer? (maksimal tiga kalimat)
Backpropagation hanya menghitung gradien loss terhadap setiap parameter dengan aturan rantai (`loss.backward()` mengisi `.grad`). Optimizer memakai gradien tersebut untuk memperbarui parameter menurut aturannya, misalnya $\theta \leftarrow \theta - \eta \nabla \theta$ pada SGD (`opt.step()`). Keduanya terpisah, sehingga gradien yang sama dapat dipakai optimizer yang berbeda.

Checklist sebelum mengumpulkan

- ☒ Identitas, seed, versi library, dan device tercantum.
- ☒ Seluruh `TODO` dan `raise NotImplementedError` sudah diganti.
- ☒ Turunan manual ditulis pada sel markdown bagian B.
- ☒ Tabel relative error memuat sembilan baris dan seluruhnya lulus ambang.
- ☒ Keempat kesalahan bagian E ditemukan, dibuktikan, dan diperbaiki bertahap.
- ☒ Notebook lolos *Restart Kernel and Run All*.
- ☒ Berkas: `M02_NIM.ipynb`, `M02_NIM.pdf`, `M02_NIM_metrics.csv`.